LeetCode – Nov 2026 | Day 2

Author: Eric Zhou
Email: eric_ds_zhou@outlook.com
Date: Nov 7, 2026

Introduction

Today, we will work on LeetCode 26 “Remove duplicates from sorted array" using `Java`. The main focus is on the Two Pointers technique, which allows us to process arrays efficiently without requiring additional space.

Leetcode 26 - Remove Duplicates from Sorted Array

Question

Given an integer array `nums` sorted in `non-decreasing order`, remove the duplicates `in-place` such that each unique element appears only `once`. The relative order of the elements should be kept the same.

Consider the number of unique elements in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of nums should contain the unique numbers in `sorted order`. The remaining elements beyond index `k - 1` can be ignored.

Intuition

The goal of this problem is to remove duplicate elements from a sorted array without using additional space.

Since the array is already sorted in non-decreasing order, duplicate elements must appear next to each other.

Therefore, we can determine whether an element is unique by comparing it with the previous element.

Instead of creating a new array, we can use the `Two Pointers` technique to overwrite duplicate elements with unique elements.

The main idea is to traverse the array while keeping track of the position where the next unique element should be stored.

Approach

First, we initialize `result = 1`, since the first element is always unique.

Then, we use a `for` loop starting from index `1` to traverse the array.

For each iteration, we compare the current element with the previous element:

$$nums[i] \neq nums[i-1]$$

If this condition is `true`, it means we have found a new unique element.

Therefore, we copy the current element to `nums[result]`:

```
nums[result] = nums[i];
```

Then, we increment `result` to indicate the next available position.

Otherwise, the current element is a duplicate, so we simply skip it and continue traversing the array.

After the loop finishes, the first `result` elements contain all unique elements in sorted order.

Finally, we return `result`, which represents the total number of unique elements.

Time Complexity:

$$\Theta(n)$$

Let n be the number of elements in the array.

Since we traverse the array only once, and each iteration performs a constant number of operations, the overall time complexity is:

$$\boxed{\Theta(n)}$$

Space Complexity:

$$\Theta(1)$$

Since we modify the original array in-place and only use a constant amount of additional space, the overall space complexity is:

$$\boxed{\Theta(1)}$$

Code

```java
class Solution {
    public int removeDuplicates(int[] nums) {
        int result = 1;
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] != nums[i-1]) {
                nums[result] = nums[i];
                result++;
            }
        }

        return result;
    }
}
```